

Ứng Dụng Tư Duy Tính Toán Trong Xây Dựng Hệ Thống Gợi Ý Phỏng Vấn

Ngành Dữ Liệu Cá Nhân Hóa Dựa Trên Knowledge Tracing (*DataTrack*)

Mai Dương Hải Đăng Tạ Gia Khiêm Nguyễn Sỹ Huỳnh Nguyễn Ngọc Nam
Trường Đại học Công nghệ Thông tin, ĐHQG-HCM
{24520253, 24520805, 24520716, 24521114}@gm.uit.edu.vn

Abstract

Báo cáo trình bày quá trình vận dụng **tư duy tính toán** (Computational Thinking – CT) để xây dựng *DataTrack*, một hệ thống luyện phỏng vấn cá nhân hóa cho ba vai trò ngành Dữ liệu: Data Analyst (DA), Data Scientist (DS) và Data Engineer (DE), tích hợp **Knowledge Tracing** (KT). Bốn trụ cột CT—phân rã, nhận diện mẫu, trừu tượng hóa, thiết kế thuật toán—được áp dụng nhất quán: nhận diện mẫu từ bộ dữ liệu (17 Knowledge Components, learning curve, mất cân bằng lớp, rủi ro rò rỉ dữ liệu); trừu tượng hóa người học thành vector năng lực $\theta \in [0, 1]^{17}$ và lịch sử thành chuỗi (q_t, r_t) ; thiết kế các thuật toán KT (BKT, DKT, SAKT) cùng cơ chế cập nhật năng lực và kiểm tra thích nghi. Question bank gồm 2.122 câu (snapshot 1.578 câu cho thực nghiệm) được xây bằng pipeline lai (thu thập + LLM + kiểm định); dữ liệu KT mô phỏng theo Item Response Theory gồm 1.000 người dùng và ~ 40.000 tương tác. Trên tập kiểm tra, mọi mô hình KT vượt baseline ($ROC - AUC \approx 0.68$ so với 0.50); DKT-Quality đạt $PR - AUC = 0.907$ và $RMSE = 0.355$, tốt nhất nhóm neural. Hệ thống hoàn chỉnh (React + FastAPI + Streamlit) với chấm điểm LLM và cơ chế dự phòng cục bộ.

1 Tổng Quan

1.1 Tổng quan

Thị trường tuyển dụng IT cạnh tranh ngày càng gay gắt; theo khảo sát TopDev (3), hơn 70% ứng viên IT tại Việt Nam chưa chuẩn bị đủ kỹ năng trước phỏng vấn. Các nền tảng hiện có (LeetCode, HackerRank) tập trung lập trình thuật toán, thiếu nội dung cho ba vai trò ngành Dữ liệu DA/DS/DE và không theo dõi được quỹ đạo kiến thức của người học theo thời gian. Câu hỏi hiệu quả nhất là câu hỏi

khó hơn năng lực hiện tại—khái niệm *zone of proximal development* (ZPD) (2).

Tư duy tính toán (1) là tập kỹ năng giúp phân tích và giải quyết vấn đề theo cách máy tính có thể thực thi, gồm bốn trụ cột: *phân rã, nhận diện mẫu, trừu tượng hóa và thiết kế thuật toán*. Báo cáo này vận dụng bốn trụ cột đó để trả lời câu hỏi: *làm sao cá nhân hóa việc luyện phỏng vấn ngành Dữ liệu theo trình độ thực tế của từng ứng viên?* Lời giải—*DataTrack*—tích hợp Knowledge Tracing (4), chấm điểm bằng mô hình ngôn ngữ lớn (LLM) (17), và giao diện web, được thiết kế xuyên suốt theo phương pháp CT.

1.2 Mục tiêu nghiên cứu

Báo cáo đặt bốn mục tiêu: (1) **nhận diện mẫu** và xây dựng bộ dữ liệu KT phù hợp cho phỏng vấn ngành Dữ liệu; (2) **trừu tượng hóa** dữ liệu thô thành biểu diễn KT chuẩn (chuỗi tương tác, vector năng lực); (3) **thiết kế thuật toán**—huấn luyện và so sánh ba họ KT (BKT, DKT, SAKT) cùng cơ chế cập nhật năng lực thời gian thực và kiểm tra thích nghi; (4) **đánh giá và tư duy phản biện**—phân tích kết quả, hạn chế và tích hợp vào ứng dụng web hoàn chỉnh.

2 Công Trình Liên Quan

Knowledge Tracing. Từ BKT (4), các mô hình KT phát triển qua DKT (5), DKVMN (14), AKT (12) và các mô hình Transformer như SAINT (13); SAKT (6) là mô hình attention đầu tiên đơn giản mà hiệu quả. Các mô hình này được kiểm chứng chủ yếu trong môi trường học thuật (MOOC, bài tập toán, ngôn ngữ).

Hệ thống học thích nghi. Knewton (15) và các Intelligent Tutoring System như ASSISTments (16) dùng KT để cá nhân hóa lộ

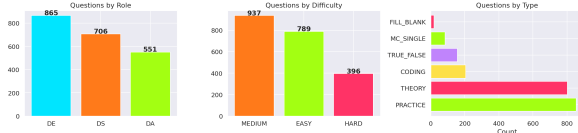


Figure 1: Phân bố câu hỏi theo vai trò và mức độ khó.

Thuộc tính	Giá trị
Người dùng	1.000 (DA/DS/DE = 334/333/333)
Câu hỏi (snapshot KT)	1.578 (triển khai: 2.122)
Knowledge Components	17
Tổng tương tác	40.140 (TB 40,1/user)
Train/Val/Test (theo user)	700/150/150
Pass rate ($Q \geq 1$)	82,5%

Table 1: Thống kê bộ dữ liệu mô phỏng.

trình học, nhưng tập trung vào nội dung học thuật.

Vị trí của *DataTrack*. Khác biệt của báo cáo là áp dụng KT cho *luyện phỏng vấn ngành Dữ liệu*—với câu hỏi mở, chấm điểm bằng LLM, nhiều loại câu hỏi và ba vai trò DA/DS/DE—một hướng còn ít được khai thác. Báo cáo cũng đề xuất biến thể *quality label* liên tục và cơ chế blend KT–EMA cho cập nhật năng lực thời gian thực.

3 Dữ Liệu

3.1 Nhận diện mẫu từ dữ liệu

Trong *DataTrack*, bài toán được nhận diện là **dự đoán chuỗi**: cho lịch sử tương tác $\{(q_1, r_1), \dots, (q_t, r_t)\}$ với q_t là câu hỏi và r_t là kết quả, dự đoán xác suất trả lời đúng câu kế tiếp $p(r_{t+1} = 1 \mid q_{t+1}, q_{1:t}, r_{1:t})$. Năng lực được theo dõi trên 17 Knowledge Components (KC) thuộc ba vai trò: DA (5 KC), DS (7 KC), DE (5 KC). Câu hỏi gồm năm loại: trắc nghiệm (MC_SINGLE), đúng/sai (TRUE_FALSE), điền chỗ trống (FILL_BLANK), lý thuyết mở (THEORY) và lập trình (CODING). Dữ liệu gồm hai phần: *question bank* (nội dung câu hỏi + metadata) và *chuỗi tương tác* của người dùng.

3.2 Nhận diện mẫu từ bộ dữ liệu *DataTrack*

Question bank dùng cho thực nghiệm KT gồm 1.578 câu (bản triển khai mở rộng lên 2.122 câu), phủ đều ba vai trò—mỗi vai trò ~ 500 –550 câu, câu MEDIUM chiếm đa số (Hình 1). Bộ dữ liệu mô phỏng gồm 1.000 người dùng và 40.140 tương tác (Bảng 1).



Figure 2: Learning curve: năng lực trung bình tăng theo vị trí trong chuỗi tương tác.

Qua phân tích khám phá (EDA), nhóm nhận diện các mẫu nổi bật: (i) **learning curve tăng đơn điệu** theo thứ tự câu hỏi (Hình 2); (ii) **mất cân bằng lớp** với pass rate 82,5%; (iii) **ma trận tương tác user–KC dạng block-diagonal** (mỗi user chỉ làm câu thuộc vai trò của mình), sparsity $\approx 77\%$ toàn cục nhưng chỉ 15–20% trong từng vai trò; (iv) **confidence bias**: tương quan giữa tự đánh giá và năng lực thực $r \approx 0.72$; (v) phân phối trình độ thực tế beginner 40,9%, intermediate 53,2%, advanced 5,9%.

3.3 Liên hệ với nguyên lý tư duy tính toán

Nguyên lý *Pattern Recognition* thể hiện rõ: các mẫu quan sát được trở thành cơ sở cho mọi quyết định kỹ thuật. Cụ thể, (a) thiếu dữ liệu thật $\rightarrow$ thiết kế **pipeline mô phỏng theo IRT**; (b) bài toán có tính **chuỗi** và tương tác giữa KC $\rightarrow$ chọn mô hình KT thay vì phân loại tĩnh; (c) **mất cân bằng lớp** $\rightarrow$ dùng PR-AUC Gain thay vì chỉ Accuracy; (d) **rủi ro rò rỉ dữ liệu** (ảnh hưởng giữa các tương tác của cùng người dùng) $\rightarrow$ chia tập **theo người dùng** thay vì theo từng tương tác; (e) mẫu **cold-start** ở giai đoạn đầu $\rightarrow$ cơ chế blend KT–EMA. Như vậy, nhận diện mẫu không chỉ mô tả dữ liệu mà còn định hình toàn bộ thiết kế hệ thống.

4 Trừu Tượng Hóa và Tiền Xử Lý Dữ Liệu

Trong CT, trừu tượng hóa là loại bỏ chi tiết không cần thiết và xây biểu diễn có cấu trúc để máy tính xử lý hiệu quả. Mô hình KT không xử lý trực tiếp “câu trả lời” dạng văn bản, mà cần các biểu diễn: năng lực người học $\theta \in [0, 1]^{17}$, lịch sử dưới dạng chuỗi cặp (q_t, r_t) , và độ khó câu hỏi quy về tham số IRT.

Tập	Người dùng	Tương tác
Train	700	28.164
Validation	150	5.911
Test	150	6.065

Table 2: Phân chia dữ liệu theo người dùng.

4.1 Xây dựng và chuẩn hóa question bank

Question bank được xây qua quy trình lai bốn giai đoạn: **thu thập** từ các kho phỏng vấn công khai (Obenner cho DE, LearningZone cho SQL, các bộ DS/DA cộng đồng), có trích dẫn nguồn; **sinh tự động** bằng pipeline đa mô hình LLM (17)—sinh câu hỏi theo KC và mức độ, tạo đáp án tham chiếu và *evaluation points*; **kiểm định** (schema, đối chiếu RAG, review thủ công); và **chuẩn hóa** các tag chi tiết về 17 KC qua bảng tra SKILL_GROUP_TO_KC (82 ánh xạ). Đây là bước trừu tượng hóa: ví dụ mọi tag SQL_WINDOW_FUNCTION, SQL_JOIN, SQL_CTE đều quy về KC SQL_DATABASE.

4.2 Biểu diễn dữ liệu

Mỗi mẫu được biểu diễn dưới dạng bộ ba (u_i, s_i, g_i) , với u_i là người dùng, $s_i = (q_t, r_t)_{t=1}^T$ là chuỗi tương tác, và g_i là định danh người dùng (group). Vì các tương tác của cùng một người dùng phụ thuộc lẫn nhau (cùng quỹ đạo năng lực), việc đưa g_i vào biểu diễn là cần thiết để tránh rò rỉ dữ liệu khi chia tập.

4.3 Phân chia tập dữ liệu

Để đánh giá khách quan, tập được chia **theo người dùng** (group) thay vì theo từng tương tác, đảm bảo mọi tương tác của một người chỉ thuộc một tập. Tỷ lệ $\approx 70:15:15$ (Bảng 2); kiểm tra xác nhận không người dùng nào xuất hiện ở nhiều tập, hạn chế rò rỉ.

4.4 Kiểm tra tính toàn vẹn dữ liệu

Trước khi huấn luyện, bộ dữ liệu được xác thực qua bảy tiêu chí: (1) *data leakage*—không người dùng nào xuất hiện ở cả train lẫn test; (2) *label distribution*—pass rate train (82,7%) $\approx$ test (82,2%), chênh $< 1\%$; (3) *sequence length* trung bình 40, phân phối đều; (4) *KC frequency drift*—2/17 KC lệch $> 3\%$ (KC 0: 3,9%; KC 3: 3,5%); (5) *class imbalance* 82,5% (do đó dùng PR-AUC); (6) *quality distribution* đều trong $[0, 1]$; (7) *cold-start coverage* 100% người dùng. Các kiểm tra này hiện thực

nguyên lý CT về xác thực dữ liệu trước khi xử lý.

4.5 Mô phỏng tương tác bằng IRT

Do thiếu dữ liệu thật, hành vi người dùng được trừu tượng hóa bằng Item Response Theory (9). Người dùng ảo được sinh với skill vector phân phối Gaussian có tương quan giữa các KC liên quan. Xác suất trả lời đúng theo IRT $P(\text{đúng}) = \sigma((\theta - b) \times 6)$; điểm chất lượng theo mô hình bậc (power-based graded response, $\alpha = 2.0$): $P(Q=2) = p^\alpha$, $P(Q=0) = (1 - p)^\alpha$; năng lực cập nhật $\theta += \eta(1 - \theta)$ khi đúng, $\theta -= \gamma\theta$ khi sai ($\eta = 0.08$, $\gamma = 0.002$, seed 42).

4.6 Chuẩn hóa toán học

Độ khó EASY/MEDIUM/HARD ánh xạ thành $b \in \{0.30, 0.55, 0.80\}$. Nhân được biểu diễn ở hai dạng: nhị phân $r_t \in \{0, 1\}$ và *quality label* liên tục $r_t \in [0, 1]$. Chuỗi tương tác được mã hóa thành tensor đầu vào cho DKT/SAKT—hoàn tất việc chuyển dữ liệu thô thành bài toán học máy chuẩn.

5 Thiết Kế Thuật Toán

5.1 Mô hình Knowledge Tracing

BKT (4) mô hình mỗi KC bằng chuỗi Markov ẩn hai trạng thái với bốn tham số $P(L_0), P(T), P(S), P(G)$; xác suất đúng $P(r_t=1) = P(L_t)(1 - P(S)) + (1 - P(L_t))P(G)$, cập nhật theo Bayes. **DKT** (5) dùng LSTM (7): $\mathbf{h}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1})$, $\hat{y}_{t+1} = \sigma(\mathbf{W}\mathbf{h}_t + \mathbf{b})$. Nhóm mở rộng **DKT-Quality** với nhân liên tục và mất mát MSE: $\mathcal{L} = \frac{1}{T} \sum_t (r_t - \hat{y}_t)^2$. **SAKT** (6) thay LSTM bằng self-attention (8), song song hóa tốt và diễn giải được attention.

5.2 Thuật toán cập nhật năng lực (EMA + Blend)

Để xử lý cold-start, năng lực được cập nhật thời gian thực bằng trung bình trượt mũ (EMA) rồi blend với KT:

$$\theta_t^{\text{EMA}} = \alpha s_t + (1 - \alpha)\theta_{t-1}^{\text{EMA}}, \quad \alpha = 0.35, \quad (1)$$

$$\hat{\theta}_t = \lambda \theta_t^{\text{KT}} + (1 - \lambda)\theta_t^{\text{EMA}}, \quad \lambda = 0.70, \quad (2)$$

trong đó KT chỉ tham gia khi lịch sử ≥ 3 tương tác. EMA (trọng số 0,3) ổn định hóa dự đoán ở giai đoạn đầu; KT (0,7) nắm động lực chuỗi. Hằng số được giữ nhất quán giữa backend và frontend để con số hiển thị đồng bộ.

5.3 Thuật toán kiểm tra thích nghi (MST)

Phiên đánh giá dùng Computerized Adaptive Testing hai giai đoạn (10): **Stage 1** chọn câu phủ đều các KC để định vị; điểm trung bình quyết định phân nhánh WEAK/MID/STRONG. **Stage 2** chọn câu chuyên sâu, ưu tiên KC yếu nhất, độ khó điều chỉnh theo nhánh. Vector năng lực cập nhật theo từng trục KC bằng cùng pipeline EMA-Blend (Công thức 2).

5.4 Engine gợi ý theo ZPD

Câu hỏi được xếp hạng theo điểm tổng hợp ưu tiên KC yếu:

$$\text{score}(q) = 0.65w + 0.30d + 0.05c, \quad (3)$$

với w là mức yếu của KC ($w = \max(0, 0.40 - \theta_{kc})/0.40$), d là độ phù hợp độ khó với năng lực (chiến lược ZPD), c là độ phủ KC. Ngưỡng KC yếu/mạnh là 0.40/0.70.

5.5 Ý nghĩa của quá trình thiết kế

Từ góc độ CT, thiết kế đã **phân rã** bài toán thành pipeline gồm các bước rõ ràng (sinh dữ liệu → huấn luyện → blend → gợi ý), và **trừu tượng hóa** mô hình KT thành ánh xạ $f(\text{lịch sử}; \theta) \rightarrow \mathbf{m}$ với $\mathbf{m}$ là mức thành thạo 17 KC. Việc đề xuất quality label minh họa khả năng chuyển một *nhận định ở mức bài toán* (điểm số mang nhiều thông tin hơn nhãn đúng/sai) thành một *cơ chế tính toán cụ thể* (đổi BCE sang MSE).

6 Đánh Giá, Kiểm Thử và Tư Duy Phản Biện

6.1 Môi trường và cấu hình

Quy trình KT offline được tổ chức thành bốn notebook chạy tuần tự—EDA → feature engineering → huấn luyện → đánh giá—mỗi bước nhận đầu ra của bước trước, cố định seed 42 để tái lập. Thực nghiệm chạy trên Kaggle (GPU Tesla T4, PyTorch (18) 2.6). Ablation study gồm 8 cấu hình: Baseline, BKT, DKT-Binary, DKT-Quality*, SAKT-Binary, SAKT-Quality*, DKT-Deep, SAKT-Deep. Mô hình neural dùng dropout (11), batch 64, 30 epoch, early stopping. Do mất cân bằng lớp 82,5%, PR-AUC Gain là chỉ số so sánh chính.

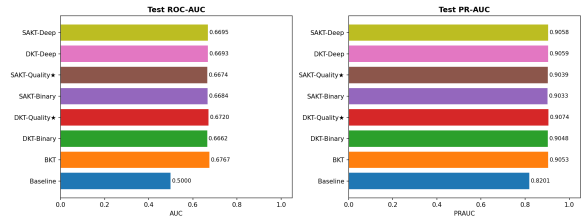


Figure 3: So sánh ROC-AUC và PR-AUC của 8 cấu hình.

6.2 Kết quả

Bảng 3 và Hình 3 cho thấy mọi mô hình KT vượt baseline ($ROC - AUC$ 0.67–0.68; PR-AUC Gain 0.083–0.087).

6.3 Phân tích kết quả

Quality label. DKT-Quality* đạt PR-AUC cao nhất (0.907) và RMSE thấp nhất (0.355), vượt DKT-Binary; nhấn liên tục cung cấp gradient phong phú hơn. Accuracy thấp hơn do tối ưu MSE thay vì cross-entropy. **Simulation bias.** BKT đạt ROC-AUC cao nhất (0.677) vì dữ liệu sinh theo IRT—cùng dạng tham số per-KC với BKT—tạo lợi thế không công bằng; trên dữ liệu thật, mô hình neural dự kiến tốt hơn. **Deep model.** Bản deep không cải thiện đáng kể nhưng tốn thời gian gấp nhiều lần: nút thắt là kích thước dữ liệu (28K tương tác), không phải kiến trúc.

6.4 Phân tích sâu và tư duy phản biện

Phân tích phân tầng (Hình 4, 5) làm rõ năm khía cạnh. **Theo KC:** hiệu suất không đồng đều—các KC tần suất thấp (KC 0 SQL_DATABASE và KC 3, drift 3,5–3,9%) có ROC-AUC thấp hơn; DKT-Quality* vượt BKT ở đa số KC của DS/DE, còn BKT cạnh tranh hơn ở KC của DA (skill DA phân bố đơn giản, hợp với dạng tham số per-KC của BKT). **Theo vị trí chuỗi:** ROC-AUC tăng từ đầu (0–25%) đến cuối chuỗi (75–100%)—càng nhiều lịch sử, dự đoán càng tốt. **Theo độ khó:** câu MEDIUM có ROC-AUC cao nhất (vùng phân biệt mạnh của IRT); EASY chịu ảnh hưởng class imbalance, HARD ít dữ liệu. **Theo loại người dùng:** nhóm intermediate dễ dự đoán nhất nhờ phân phối đúng/sai cân bằng hơn. **Cold-start:** tương quan dương yếu giữa cold-start score và per-user ROC-AUC, cho thấy self-rating mang thông tin bổ sung nhưng không phải yếu tố duy nhất. Phân tích

Mô hình	ROC-AUC	PR-AUC	PR-AUC Gain	RMSE	Acc	Thời gian (s)
Baseline	0.500	0.820	0.000	0.384	0.820	0.0
BKT	0.677	0.905	0.085	0.374	0.820	61.9
DKT-Binary	0.666	0.905	0.085	0.375	0.822	30.8
DKT-Quality*	0.672	0.907	0.087	0.355	0.796	20.9
SAKT-Binary	0.668	0.903	0.083	0.373	0.822	11.3
SAKT-Quality*	0.667	0.904	0.084	0.356	0.776	12.0
DKT-Deep	0.669	0.906	0.086	0.375	0.822	116.4
SAKT-Deep	0.670	0.906	0.086	0.373	0.822	31.4

Table 3: Kết quả ablation study trên tập kiểm tra (150 người dùng, 6.065 tương tác). *: dùng quality label liên tục.

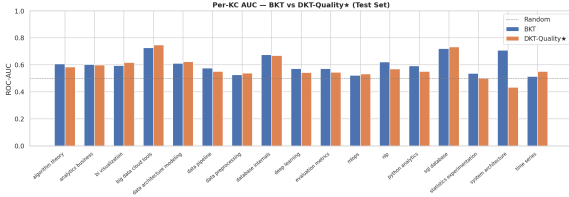


Figure 4: ROC-AUC theo từng KC: BKT vs DKT-Quality*.

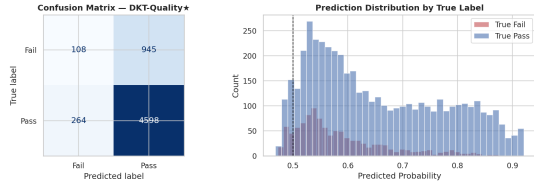


Figure 5: Confusion matrix và phân phối điểm dự đoán—DKT-Quality*.

lỗi cho thấy mô hình thiên về dự đoán Pass (False Negative > False Positive) do mất cân bằng lớp. Những quan sát này thể hiện tư duy phản biện: một phần kết quả tốt đến từ thiên lệch mô phỏng, nên cần kiểm chứng trên dữ liệu thật.

6.5 Hệ thống ứng dụng

DKT-Quality* được chọn làm mô hình triển khai vì dự đoán cả 17 KC trong một forward pass, phù hợp gợi ý thời gian thực. Hệ thống tổ chức thành ba tầng. **Tầng dữ liệu** chuẩn hóa question bank và ánh xạ `skill_groups` về 17 KC qua Competency Engine. **Tầng KT/scoring** nạp mô hình tốt nhất ở runtime, chạy pipeline EMA-Blend (Công thức 2) và chấm điểm. **Tầng phục vụ** gồm FastAPI (9 endpoint), Streamlit làm host và React SPA; FastAPI được tách khỏi Streamlit để phục vụ tính đúng MIME type và gọi API độc lập. Câu khách quan chấm tất định; câu tự luận/coding chấm bằng Gemini (17) theo rubric, lui về lo-

cal rubric khi LLM lỗi (một *evaluation point* “đạt” khi $\geq 50\%$ từ khóa xuất hiện). Toàn bộ luồng onboarding → kiểm tra thích nghi (MST hai giai đoạn) → chấm điểm → cập nhật năng lực → dashboard hoạt động end-to-end.

7 Kết Luận

Báo cáo đã vận dụng đầy đủ bốn trụ cột tư duy tính toán để xây dựng *DataTrack*: nhận diện mẫu từ bộ dữ liệu, trừu tượng hóa người học thành chuỗi tương tác và vector năng lực, thiết kế các thuật toán KT cùng cơ chế cập nhật/gợi ý, và đánh giá có phản biện. Kết quả xác nhận KT mang lại giá trị thực ($ROC - AUC \approx 0.68$ so với 0.50), với DKT-Quality tốt nhất nhóm neural.

7.1 Hạn chế

Simulation bias (nghiêm trọng nhất): dữ liệu IRT tạo lợi thế cho BKT, cần dữ liệu thật để kiểm chứng. **Sequence length cố định** 40 do pool câu mỗi vai trò bị cap, làm mô hình không học được chuỗi dài/ngắn khác nhau. **Mất cân bằng lớp** (pass rate $\approx 82\%$) làm PR-AUC bị inflate; PR-AUC Gain đáng tin hơn để so sánh. **KC frequency imbalance** (nhẹ): 2/17 KC lệch tần suất train/test > 3%, khắc phục bằng stratified split theo KC. **Data scale** (nhẹ): $\sim 28K$ tương tác train chưa đủ để khai thác mô hình sâu. **Phân phối trình độ lệch** (advanced chỉ 5,9%) và **phụ thuộc LLM** khi chấm câu mở—chưa calibrate với human annotation.

7.2 Hướng phát triển

Thu thập dữ liệu thật và A/B test; tích hợp forgetting curve (19) và spaced repetition (20); thử AKT (12)/SAINT (13) khi đủ dữ liệu; mở rộng question bank và triển khai production.

Lời cảm ơn

Nhóm xin cảm ơn giảng viên môn Tư duy tính toán và quý thầy cô Trường Đại học Công nghệ Thông tin, ĐHQG-HCM đã hướng dẫn và hỗ trợ trong quá trình thực hiện.

References

- [1] J. M. Wing. 2006. Computational thinking. *Communications of the ACM*, 49(3):33–35.
- [2] L. S. Vygotsky. 1978. *Mind in Society*. Harvard University Press.
- [3] TopDev. 2024. Vietnam IT Market Report 2024. Technical report.
- [4] A. T. Corbett and J. R. Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278.
- [5] C. Piech et al. 2015. Deep knowledge tracing. In *NeurIPS*, volume 28.
- [6] S. Pandey and G. Karypis. 2019. A self-attentive model for knowledge tracing. In *Proc. EDM*.
- [7] S. Hochreiter and J. Schmidhuber. 1997. Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- [8] A. Vaswani et al. 2017. Attention is all you need. In *NeurIPS*.
- [9] S. E. Embretson and S. P. Reise. 2000. *Item Response Theory for Psychologists*. Lawrence Erlbaum.
- [10] H. Wainer et al. 2000. *Computerized Adaptive Testing: A Primer*, 2nd ed. Lawrence Erlbaum.
- [11] N. Srivastava et al. 2014. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15(56):1929–1958.
- [12] A. Ghosh, N. Heffernan, and A. S. Lan. 2020. Context-aware attentive knowledge tracing. In *ACM SIGKDD*.
- [13] Y. Choi et al. 2020. Towards an appropriate query, key, and value computation for knowledge tracing. In *ACM L@S*.
- [14] J. Zhang, X. Shi, I. King, and D.-Y. Yeung. 2017. Dynamic key-value memory networks for knowledge tracing. In *Proc. WWW*, pages 765–774.
- [15] P. Ferreira. 2017. Knewton’s adaptive learning platform. White Paper, Knewton Inc.
- [16] N. T. Heffernan and C. L. Heffernan. 2014. The ASSISTments ecosystem. *International Journal of Artificial Intelligence in Education*, 24(4):470–497.
- [17] Gemini Team, Google. 2023. Gemini: A family of highly capable multimodal models. arXiv:2312.11805.
- [18] A. Paszke et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*.
- [19] H. Ebbinghaus. 1885. *Memory: A Contribution to Experimental Psychology*.
- [20] P. A. Wozniak and E. J. Gorzelanczyk. 1990. Optimization of repetition spacing in the practice of learning. *Acta Neurobiologiae Experimentalis*, 54:59–62.